Лекция 2: Отношения между объектами. Ассоциация, агрегация, композиция и наследование. Принцип «Composition over Inheritance»

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Structure → Order (От разрозненных классов к устойчивым объектным связям)

Введение и инженерный контекст

На первой лекции мы рассмотрели, как инкапсулировать состояние и поведение в рамках отдельного класса и защитить его инварианты. Однако реальные программные комплексы состоят из сотен взаимодействующих объектов.
Ключевой архитектурный вопрос: **как правильно связать объекты между собой?**
В классической литературе 1990-х годов упор делался на наследование как на главный механизм повторного использования кода. Однако три десятилетия индустриальной практики доказали: **злоупотребление наследованием — одна из самых частых причин архитектурного кризиса кодовой базы**. Изменения в базовом классе каскадно ломают поведение десятков наследников (проблема хрупкого базового класса — *Fragile Base Class Problem*).
В этой лекции мы детально разберем типологию отношений между объектами и фундаментальный принцип объектно-ориентированного проектирования: **«Composition over Inheritance» (Композиция предпочтительнее наследования)**.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Типология связей: Зависимость (Uses-A), Ассоциация (Has-A), Агрегация и Композиция	Архитектурная классификация
2	00:20–00:40	Наследование (Is-A): польза, ограничения и проблемы (Fragile Base Class, Class Explosion)	Разбор антипаттернов

№	Время	Тема	Формат
3	00:40–01:05	Принцип «Composition over Inheritance»: компонентная модель поведения	Теория + код
4	01:05–01:25	Реализация на Python 3.12: моделирование системы тарификации и платежей	Живой код
5	01:25–01:30	Итоги лекции, контрольные вопросы и связь с лабораторной работой	Рефлексия

---

1. Типология отношений между классами

В объектном моделировании (и диаграммах UML) выделяют 4 основных типа отношений:

```
[ Зависимость (Dependency) ]      --> Слабейшая связь (USES-A)
          |
[ Ассоциация (Association) ]      --> Знакомство объектов
          |
[ Агрегация (Aggregation) ]       --> Владение со слабым жизненным циклом (HAS-A)
          |
[ Композиция (Composition) ]      --> Неразрывное владение (PART-OF / HAS-A)
          |
[ Наследование (Inheritance) ]    --> Строгая специализация (IS-A)
```

1.1. Зависимость (Dependency: Uses-A)

Класс *A* кратковременно использует класс *B* в качестве аргумента метода или локальной переменной. Изменение интерфейса *B* затрагивает *A*, но *A* не хранит ссылку на *B* в своих полях.

```python
class ReceiptPrinter:
    def print_ticket(self, ticket: SingleTripTicket) -> None:
        # ReceiptPrinter кратковременно использует ticket (Uses-A)
        print(f"--- БИЛЕТ № {ticket.ticket_code} ---")
```

1.2. Ассоциация (Association: Has-A)

Класс *A* хранит ссылку на экземпляр класса *B* в качестве поля. Объекты равноправны, их жизненные циклы не зависят друг от друга.

```python
class Driver:
    def __init__(self, name: str, license_id: str):
        self.name = name
        self.license_id = license_id
```

```python
class Bus:
    def __init__(self, plate_number: str):
        self.plate_number = plate_number
        self.driver: Driver | None = None # Ассоциация: водитель может смениться
```

1.3. Агрегация vs Композиция (Жизненный цикл владения)

Это критическое различие, часто вызывающее путаницу:

- **Агрегация (Aggregation)**: отношение «целое — часть», где часть может существовать независимо от целого. Жизненные циклы разделены.
    - *Пример*: Автобус (Bus) и Маршрут (Route). Если маршрут временно закрывается или отменяется, автобусы физически не уничтожаются — они перенаправляются на другие маршруты.
- **Композиция (Composition)**: строгое отношение владения. Часть не имеет смысла и не может существовать вне контекста владельца. Уничтожение целого ведет к уничтожению его частей.
    - *Пример*: Транзакция (Transaction) и её финансовые записи (LedgerEntry). Бухгалтерская запись списания не имеет смысла в отрыве от породившей её транзакции.

```python
# Композиция: Двигатель создается внутри Автобуса и умирает вместе с ним
class Engine:
    def __init__(self, serial: str, power_kw: int):
        self.serial = serial
        self.power_kw = power_kw

class ElectricBus:
    def __init__(self, plate_number: str, engine_serial: str):
        self.plate_number = plate_number
        # КОМПОЗИЦИЯ: создание и владение внутри контейнера
        self._engine = Engine(engine_serial, power_kw=250)
```

---

2. Наследование (Is-A) и ловушка комбинаторного взрыва

Наследование декларирует отношение «является» (*Is-A*). Подкласс расширяет функционал родителя или переопределяет его методы.

Проблема комбинаторного взрыва классов (Class Explosion)

Представьте систему тарифов городского транспорта. У нас есть: - Базовый тариф (StandardFare) - Льготный студенческий тариф (скидка 50%) - Зональный пригородный тариф (наценка 30%) - Ночной тариф (наценка 20%)
Если мы попытаемся смоделировать все комбинации через наследование:

```
Fare
├── StandardFare
├── StudentFare
├── ZoneFare
│   └── StudentZoneFare
│       └── StudentZoneNightFare (!!!)
└── NightFare
    └── NightZoneFare
```

При добавлении ещё одного модификатора (например, «Праздничный день» или «Оплата через QR») количество классов будет расти экспоненциально: 2^N. Кодовая база превращается в непроходимый лабиринт дублирующегося кода.

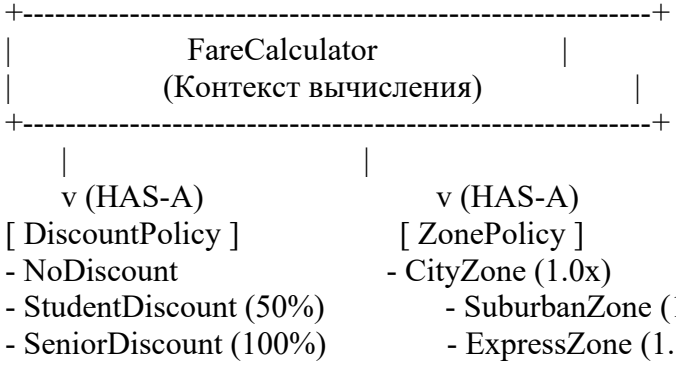
---

3. Принцип «Composition over Inheritance»

Вместо того чтобы кодировать различные аспекты поведения в жесткой иерархии типов, мы **декомпозируем поведение на независимые компоненты** и внедряем их в класс через композицию.

Формула композиции:

Не говорите, чем объект *является* (через дерево наследования).
Сконфигурируйте объект из того, чем он *обладает* и что он *умеет делать*.



Теперь любое новое правило тарификации — это просто новый легковесный класс, реализующий конкретную политику. Мы можем комбинировать их во время выполнения программы (Runtime), чего наследование сделать не позволяет в принципе!

---

4. Проектирование гибкой тарифной системы на Python 3.12

```python
# domain/pricing.py
from decimal import Decimal
from typing import Protocol


class DiscountPolicy(Protocol):
    """Протокол расчета скидки."""
    def apply_discount(self, amount: Decimal) -> Decimal:
        ...


class ZoneMultiplierPolicy(Protocol):
    """Протокол расчета зонального коэффициента."""
    def get_multiplier(self) -> Decimal:
        ...


# Конкретные реализации политик скидок (легковесные компоненты)
class NoDiscountPolicy:
    def apply_discount(self, amount: Decimal) -> Decimal:
```

```
        return amount


class StudentDiscountPolicy:
    def apply_discount(self, amount: Decimal) -> Decimal:
        return amount * Decimal("0.50")


class SeniorDiscountPolicy:
    def apply_discount(self, amount: Decimal) -> Decimal:
        return Decimal("0.00") # Бесплатный проезд


# Конкретные реализации зональных коэффициентов
class CityZonePolicy:
    def get_multiplier(self) -> Decimal:
        return Decimal("1.00")


class SuburbanZonePolicy:
    def get_multiplier(self) -> Decimal:
        return Decimal("1.35") # +35% за пригород


# Главный вычислитель тарифа: использует КОМПОЗИЦИЮ
class FareCalculator:
    """
    Калькулятор стоимости проезда.
    Комбинирует базовую ставку, политику скидки и зональную политику.
    """
    def __init__(
        self,
        base_rate: Decimal,
        discount_policy: DiscountPolicy,
        zone_policy: ZoneMultiplierPolicy
    ):
        if base_rate <= Decimal("0.00"):
            raise ValueError("Базовая ставка должна быть положительной.")

        self._base_rate = base_rate
        self._discount_policy = discount_policy
        self._zone_policy = zone_policy

    def calculate_fare(self) -> Decimal:
        """
        Итоговый расчет: (Базовый тариф * Зональный коэффициент) -> Применение скидк
и.
        """
        zoned_amount = self._base_rate * self._zone_policy.get_multiplier()
        final_amount = self._discount_policy.apply_discount(zoned_amount)
        # Округляем до двух знаков после запятой
        return final_amount.quantize(Decimal("0.01"))
```

Демонстрация гибкости в клиентском коде:

```
# Городской студент
student_city = FareCalculator(
    base_rate=Decimal("100.00"),
    discount_policy=StudentDiscountPolicy(),
    zone_policy=CityZonePolicy()
)
print("Студент в городе:", student_city.calculate_fare()) # 50.00 KZT

# Студент в пригороде (Каскелен / Талгар)
student_suburban = FareCalculator(
    base_rate=Decimal("100.00"),
    discount_policy=StudentDiscountPolicy(),
    zone_policy=SuburbanZonePolicy()
)
print("Студент в пригороде:", student_suburban.calculate_fare()) # 67.50 KZT
```

Результат: 0 дополнительных подклассов, 100% повторное использование кода, возможность менять тарифы прямо на лету!

Контрольные вопросы и выводы

1. В каких случаях отношение между классами следует моделировать как агрегацию, а в каких — как композицию?
2. Почему глубокие иерархии наследования (3+ уровней) считаются архитектурным антипаттерном?
3. В чем суть проблемы «хрупкого базового класса» (*Fragile Base Class Problem*)?
4. Как принцип «Composition over Inheritance» позволяет избежать комбинаторного взрыва классов?
5. Почему композиция позволяет менять поведение объекта во время выполнения (Runtime), в отличие от наследования?

Литература к лекции

1. **Гамма Э. и др.** Приемы объектно-ориентированного проектирования. Паттерны проектирования. – СПб.: Питер, 2020. (Глава 1: Принципы объектно-ориентированного проектирования: «Применяйте композицию объектов вместо наследования классов»).
2. **Мартин Р.** Чистая архитектура. – СПб.: Питер, 2021. (Глава 11: Принцип разделения интерфейсов).
3. **Блох Дж.** Effective Java. – 3rd ed. – Addison-Wesley, 2018. (Item 18: Favor composition over inheritance).